



Inteligencia Artificial
Grado en Ingeniería Informática en Sistemas de Información
ENSEÑANZAS PRÁCTICAS Y DE DESARROLLO
EPD2: Machine Learning – Regresión Lineal

Objetivos

- Implementación en Python de un algoritmo de Regresión Lineal univariable y multivariable y su aplicación a datos.

Bibliografía Básica

Machine Learning. Tom Mitchell. MacGraw-Hill, 1997

Ejercicios

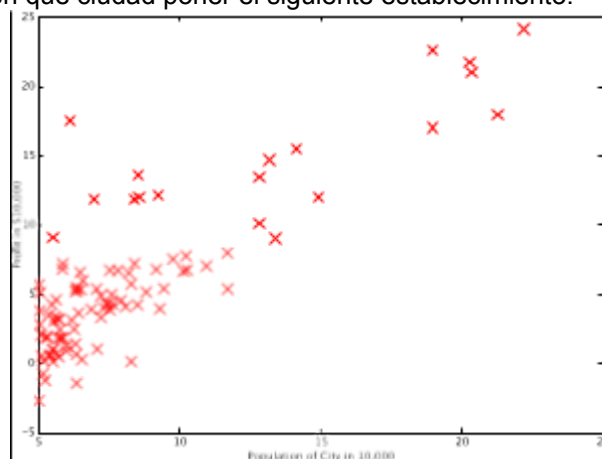
Univariable

Implementar y aplicar regresión lineal con una variable para predecir los beneficios de un restaurante ambulante. Ponte en el papel del Director Ejecutivo de una franquicia de este tipo de restaurantes y estás considerando diferentes ciudades para la apertura de un nuevo establecimiento. La cadena ya cuenta con restaurantes en varias ciudades, y dispone de los datos de los beneficios y las poblaciones de las ciudades. Deberías utilizar los datos como ayuda para seleccionar en qué ciudad poner el siguiente establecimiento.

El fichero ex1data1.txt contiene los datos para nuestro problema de regresión lineal. La primera columna es la población de una ciudad y la segunda es el beneficio de un establecimiento en esa ciudad. Un valor negativo de beneficio indica una pérdida.

Utilizar el script main.py para ir incorporando el código y las llamadas a las funciones que se piden en los siguientes ejercicios.

EJ1. Normalmente, antes de empezar cualquier tarea se suele visualizar el conjunto de datos. Añadir al fichero main.py el código necesario para cargar los datos, e implementar la función plotData(X,y) de plotData.py para representar los datos gráficamente. A la derecha se muestra un ejemplo del resultado de la visualización.



EJ2. El objetivo de la regresión lineal es minimizar el coste de la función:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Implementar la función computeCost(X,y,theta) de manera que devuelva el valor del coste. La matriz X es la matriz formada por la población de cada ciudad e y es el vector formado por los beneficios del establecimiento de cada ciudad. Para facilitar las operaciones vectorizadas, añade una columna con todos sus elementos a 1 a la matriz X como primera columna, de esta manera se podrá calcular la hipótesis h que viene dada por el modelo lineal:

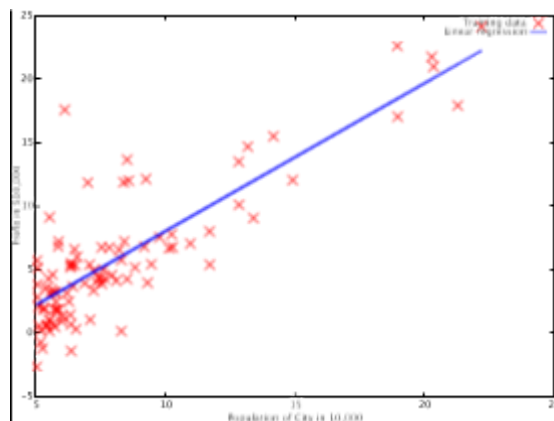
$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

Para comenzar inicializar los parámetros theta a 0. El resultado del coste inicial debe ser 32,07.

EJ3. Implementar el método de descenso del gradiente (ver documento de EB, p. 42). La función recibirá como parámetro, además de los datos X e y, los parámetros theta, alpha y el número de iteraciones. Estos dos últimos se pueden inicializar a 0,01 y 1500 respectivamente. La función deberá devolver los parámetros theta finales y un histórico con el coste en cada iteración. Mostrar por pantalla los valores de theta.

Utilizar la función `plotIterationsVsCost()` proporcionada en el fichero [plotData.py](#) para comprobar que el descenso del gradiente para los parámetros indicados funciona correctamente. Se representará el número de iteraciones vs el coste.

EJ4. Representar gráficamente los resultados obtenidos y calcular la predicción de beneficios para una población de 35000 habitantes y para otra de 70000 habitantes. Mantener la figura anterior para poder comprobar la diferencia con los datos originales como en la figura de la derecha.



Multivariable

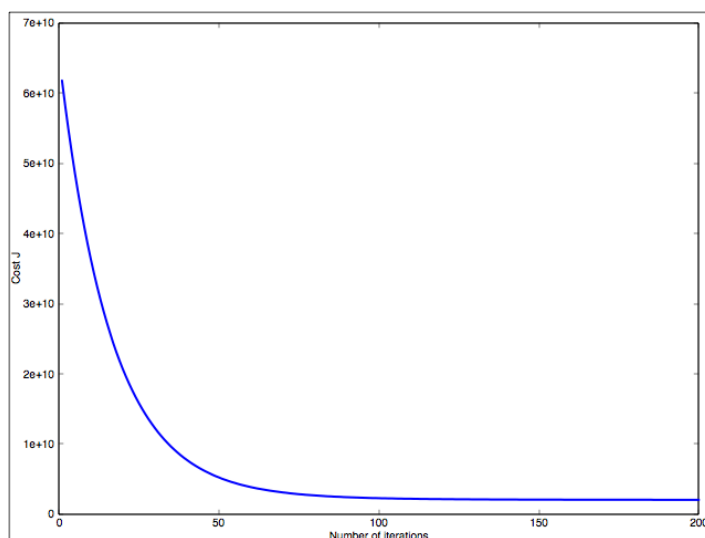
Implementar regresión lineal con múltiples variables para predecir los precios de casas. Suponer que queremos vender nuestra casa y queremos saber cuál sería un buen precio de mercado. Una opción sería recolectar información de ventas de casas recientes y crear un modelo de precios de casas.

El fichero `ex1data2.txt` contiene un conjunto de entrenamiento de precios de casas en una ciudad estadounidense. La primera columna corresponde con el tamaño de la casa (en pies cuadrados), la segunda columna indica el número de dormitorios, y la tercera el precio de la casa.

Puede reutilizar código del ejercicio univariable.

Antes de nada, muestra las diez primeras filas de datos y a continuación:

1. Implementar una función para normalizar los datos de la matriz X que utilice la desviación estándar.
2. Si la función de coste implementada en la práctica anterior no se puede utilizar para múltiples variables, codificar una nueva. En este caso también hay que añadir una primera columna a la matriz X con todos sus elementos a 1.
3. Lo mismo ocurre con el método del gradiente, ver si se puede reutilizar o hay que implementar uno nuevo.
4. Mostrar el gráfico de convergencia que se observa en la figura obtenido con alpha igual a 0.03 y 200 iteraciones.
5. Predecir el precio de una casa de 1650 pies cuadrados y 3 habitaciones.
6. Calcular los parámetros theta mediante el uso de la ecuación normal, y predecir nuevamente el precio de la vivienda con los valores anteriores.



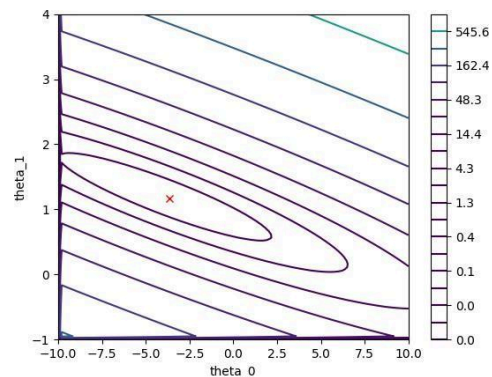
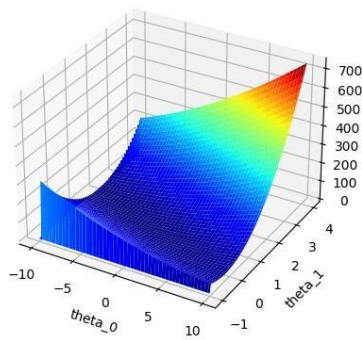


Problemas

P1. En el caso de estudio utilizado en los ejercicios anteriores, probar con diferentes valores de α (0.1, 0.03, 0.01, ...).

Representar y comparar los diferentes resultados obtenidos con la función `plotIterationsVsCost()` para comprobar qué valores de α son válidos y cuáles no. Finalmente, utilizar los mejores valores de θ para predecir de nuevo los beneficios para una población de 35000 y 70000 habitantes. Curiosidad: realizar también ambas predicciones utilizando diferentes modelos de regresión lineal de la librería `sklearn` (`sklearn.linear_model`) como por ejemplo: `LinearRegression()`, `Ridge()` o `Lasso()`.

P2. Incluir el código necesario en `main.py` para obtener las gráficas de la función de coste que se muestran a continuación. Para ello, utilizar la función `plotData_cost()` proporcionada que hace uso de los comandos `plot_surface` y `contour`.



P3. Utilizar la regresión lineal para predecir los precios de casas. Suponer que queremos vender nuestra casa y queremos saber cuál sería un buen precio de mercado. Una opción sería recolectar información de ventas de casas recientes y crear un modelo de precios de casas.

El fichero `ex1data2.txt` contiene un conjunto de entrenamiento de precios de casas en una ciudad estadounidense. La primera columna corresponde con el tamaño de la casa (en pies cuadrados), la segunda columna indica el número de dormitorios, y la tercera el precio de la casa.

Utilizando la función de coste y el método de descenso del gradiente antes implementados, indica cuál de los dos atributos determina mejor el precio de la casa. Considerar los valores 0,03 y 200 para α y el número de iteraciones respectivamente. Como en el ejercicio 4, visualizar los resultados obtenidos con cada una de las opciones o atributos.

P4. En el caso de estudio utilizado en los ejercicios anteriores, probar con diferentes valores de α , como por ejemplo 0.3, 0.1, 0.03, 0.01,... usando la normalización de los datos con desviación estándar (Ejercicio 1). Representar los gráficos de convergencia (Ejercicio 4) y comparar los diferentes resultados obtenidos. Finalmente, utilizar los mejores valores de θ para predecir el precio de una casa de 1650 pies cuadrados y 3 habitaciones. Además, se podrían comparar estos resultados con los obtenidos en la ampliación de problemas de la práctica anterior aplicando regresión lineal con una sola variable.

P5. Se va a utilizar la librería `scikit learn` (también conocida como `sklearn`) para dividir en dos partes los conjuntos de datos `X` e `y` de `ex1data2.txt`: una parte será el conjunto de entrenamiento y otro de test, conteniendo el primero el 70% de los ejemplos elegidos aleatoriamente, mientras que el segundo contendrá el resto. Para ello utilizar la función `train_test_split()`.

Al finalizar deberías tener `Xtrain`, `Xtest`, `ytrain` e `ytest`.

A continuación, averiguar los valores de θ con los nuevos conjuntos de entrenamiento (partiendo de la inicialización de θ a 0 y el α que se haya considerado mejor en el problema anterior) y comparar los errores obtenidos mediante las medidas error cuadrático medio y correlación sobre los conjuntos de test usando tanto las funciones que se han implementado como la "LinearRegression" de `sklearn`. Se pueden usar las medidas de error que se encuentran en la librería `sklearn`: `sklearn.metrics.mean_squared_error(y_test, y_pred)` y `sklearn.metrics.r2_score(y_test, y_pred)` respectivamente.

